Question 16: Java program for multiplication of two numbers using Hadoop MapReduce

using Hadoop MapReduce

Aim

To multiply two integer numbers using a Hadoop MapReduce program.

Procedure

1. Create an input file containing the two integer values separated by a comma.

2. The mapper reads the input line, extracts the two numbers, multiplies them and emits the product with a fixed key.

3. The reducer receives the product value and writes it to the output.

4. Compile the Java source file, package it into a JAR and upload the input file to HDFS.

5. Run the Hadoop job and display the output file.

Program

```
import java.io.IOException;

import org.apache.hadoop.conf.Configuration;

import org.apache.hadoop.fs.Path;

import org.apache.hadoop.io.*;

import org.apache.hadoop.mapreduce.*;

import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;

import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;

public class MultiplyNumbers {

public static class M extends Mapper<LongWritable,Text,Text,IntWritable> {

public void map(LongWritable key, Text value, Context c)

throws IOException, InterruptedException {

String[] n = value.toString().trim().split(",");

int a = Integer.parseInt(n[0].trim());

int b = Integer.parseInt(n[1].trim());

c.write(new Text("Product"), new IntWritable(a * b));

}

}

public static class R extends Reducer<Text,IntWritable,Text,IntWritable> {
```

```
public void reduce(Text key, Iterable<IntWritable> values, Context c)
throws IOException, InterruptedException {
for (IntWritable v : values)
c.write(key, new IntWritable(v.get()));
}
}
public static void main(String[] args) throws Exception {
Job job = Job.getInstance(new Configuration(), "Multiply Two Numbers");
job.setJarByClass(MultiplyNumbers.class);
job.setMapperClass(M.class);
job.setReducerClass(R.class);
job.setOutputKeyClass(Text.class);
job.setOutputValueClass(IntWritable.class);
FileInputFormat.addInputPath(job, new Path(args[0]));
FileOutputFormat.setOutputPath(job, new Path(args[1]));
System.exit(job.waitForCompletion(true) ? 0 : 1);
}
}
```

Input file

```
# numbers.txt
10,20
```

Compile and execute

```
export HADOOP_CLASSPATH=$(hadoop classpath)
javac -classpath "$HADOOP_CLASSPATH" -d classes MultiplyNumbers.java
jar -cvf multiply.jar -C classes .
hdfs dfs -mkdir -p /input/multiply
hdfs dfs -put numbers.txt /input/multiply/
hadoop jar multiply.jar MultiplyNumbers /input/multiply /output/multiply
hdfs dfs -cat /output/multiply/part-r-00000
```

Output of Program

```
Product 200
```

Result

The Hadoop MapReduce program successfully multiplies 10 and 20 and displays the product 200.

LAB TEST EXECUTION CHECKLIST

• Start HDFS before running HDFS, Hive or HBase operations.

• Use a new output directory for every MapReduce run; Hadoop normally fails if the output directory already exists.

• Before rerunning a job, remove its old output directory with hdfs dfs -rm -r /output/jobname.

• Use the exact Java, Hadoop, Hive and HBase versions installed in the lab; their compatibility matters.

• Use hdfs dfs -ls and hdfs dfs -cat to verify HDFS files and MapReduce output.

• For Java MapReduce programs, compile with the Hadoop classpath and include the correct main class in the hadoop

jar command.

• Sample output shown in this guide is expected output for the sample input, not a live execution log.

Common MapReduce command pattern

export HADOOP_CLASSPATH=$(hadoop classpath)

javac -classpath "$HADOOP_CLASSPATH" -d classes ProgramName.java

jar -cvf program.jar -C classes .

hdfs dfs -rm -r -f /output/result

hadoop jar program.jar MainClass /input/path /output/result

hdfs dfs -cat /output/result/part-r-00000

Prepared from the uploaded CCS334 – Big Data Analytics (Regulation 2021) question paper. The paper lists Questions 1–17;

Questions 12, 15 and 17 have intentionally been excluded as requested.